uMAC Utilities, Enabling Appcasting, and On-Demand Appcasting

AppCasting and uMAC utilities.

Included MIT MOOS-IvP PDF sources:

- app_appcasting_umactools.pdf
- app_appcasting_extend.pdf
- app_appcasting_under_hood.pdf

The uMAC Utilities

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1 Overview

In this section the utilities for viewing appcasts and/or realmcasts are discussed. They include:

- The `uMACView` utility: A GUI tool containing navigation tools for viewing appcasts or realmcasts across multiple vehicles or nodes. A snapshot is shown in Figure 1.
- The `uMAC` utility: A utility implement in the terminal window. Simpler in the interface, but notable in that it allows a user to remotely launch the tool on deployed vehicle.
- `uMACView` integrated with `pMarineViewer`: An interface nearly identical with `uMACView` fully integrated within `pMarineViewer`, convenient for those already using `pMarineViewer`.

Note: Starting with the first release after 2019's Release 19.8, `uMACView` and `pMarineViewer` were augmented to include realmcasting in addition to appcasting. The term *infocasting* is the general term referring to either appcasting or realmcasting. To use realmcasting, an additional app, `pRealm` needs to be running in each MOOS community, typically on the shoreside and in each vehicle community. More information on `pRealm` can be found in [1]

The `uMAC` utility currently only support appcasting output.

2 The uMACView Utility

The uMACView utility is an appcast or realmcast viewer with a graphical user interface using the FLTK library. It contains three panes as shown in Figure 1. The bottom pane renders incoming appcasts or realmcasts. The top right pane lists possible applications for selecting appcast or realmcast viewing for the present node. The upper left pane shows all presently known nodes from which appcasts or realmcasts have been received. In realmcasting, the upper left pane may also offer a number of variable clusters to select from.

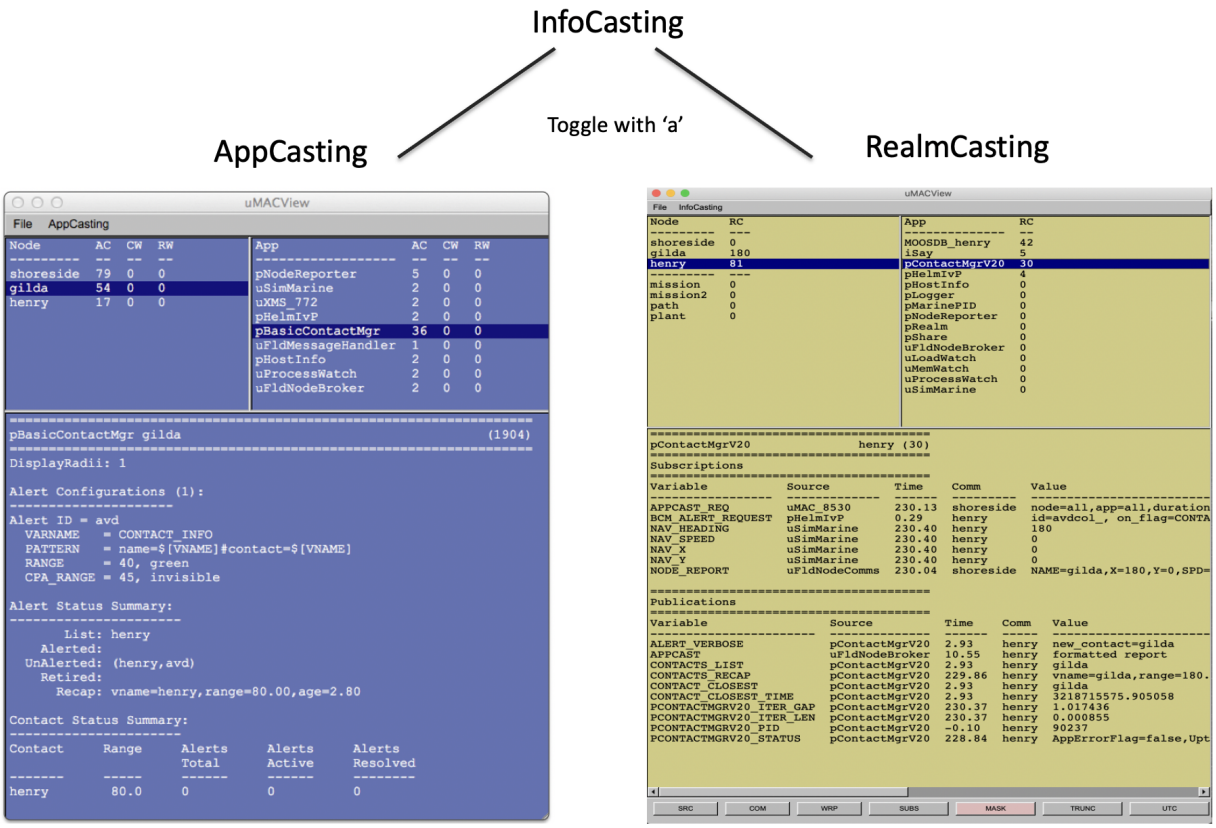

Figure 1: The uMACView utility receives appcasts or realmcasts from perhaps several different nodes and applications on each node. The interface allows the user to select a node and application for viewing the selected application's latest appcast or realmcast. The with appcasting, the viewer will raise alerts from non-selected nodes and applications when or if a run warning occurs. To toggle between appcasting and realmcasting, the `'a'` key is used.

The content mode is either *appcasting* or *realmcasting*. To toggle between the two modes, the `'a'` key is used. Alternatively the startup mode can be set with the `content_mode` configuration parameter, which is set to either `appcast` or `realmcast`, with the former being the default.

2.1 AppCasting

The content of the *appcast pane* is solely determined by the appcast content itself. From the viewer's perspective it is simply pushing a list of strings to a browser pane. Even the formatting of the

header lines showing the application name, node name and application iteration, are formatted by a method defined over the AppCast class.

The content of the `application pane` lists the applications known thus far to the viewer from incoming appcasts for a particular node. The order is shown by the order in which they were received. The three columns, `"AC"`, `"CW"`, and `"RW"`, show the number of appcasts received from the application, and the number of run warnings and configuration warnings in the latest appcast.

The content of the *node pane* in the upper left lists the nodes known thus far to the viewer from incoming appcasts. The order is shown by the order in which they were received. The three columns, `"AC"`, `"CW"`, and `"RW"`, show the number of appcasts received from a given node for *all* applications from that node, as well as the sum of all run warnings and configuration warnings for all applications received from the given node.

2.2 RealmCasting

The content of the *realmcast pane* is fed from an app called `pRealm` running on the node selected in the node pane, and related to the application selected in the upper right app pane. A typical realmcast report is shown in Figure 2:

```
=====================================
pContactMgrV20              gilda (140)
=====================================
Subscriptions
=====================================
Variable           Source         Time    Comm       Value
-----------------  -------------  ------  ---------  ------------------------------------------------------------------------------------------
APPCAST_REQ        uMAC_220       241.52  shoreside  node=all,app=all,duration=3.0,key=uMAC_220:app,thresh=run_warning
BCM_ALERT_REQUEST  pHelmIvP       0.28    gilda      id=avdcol_, on_flag=CONTACT_INFO=name=$[VNAME] # contact=$[VNAME],alert_range=80, cpa_rang
                                                     e=85,match_type=mokai
NAV_HEADING        uSimMarine     242.21  gilda      180
NAV_SPEED          uSimMarine     242.21  gilda      0
NAV_X              uSimMarine     242.21  gilda      180
NAV_Y              uSimMarine     242.21  gilda      0
NODE_REPORT        uFldNodeComms  241.73  shoreside  NAME=henry,X=0,Y=0,SPD=0,HDG=180,TYPE=kayak,MODE=PARK,ALLSTOP=ManualOverride,INDEX=479,TIM
                                                     E=3218502913.32,LENGTH=4

=====================================
Publications
=====================================
Variable                 Source          Time    Comm   Value
-----------------------  --------------  ------  -----  ------------------------------------------------------------------------------------------
ALERT_VERBOSE            pContactMgrV20  2.91    gilda  new_contact=henry
APPCAST                  uProcessWatch   7.68    gilda  formatted report
CONTACTS_LIST            pContactMgrV20  2.91    gilda  henry
CONTACTS_RECAP           pContactMgrV20  242.05  gilda  vname=henry,range=180.00,age=0.69
CONTACT_CLOSEST          pContactMgrV20  2.91    gilda  henry
CONTACT_CLOSEST_TIME     pContactMgrV20  2.91    gilda  3218502674.874138
PCONTACTMGRV20_ITER_GAP  pContactMgrV20  242.04  gilda  1.004112
PCONTACTMGRV20_ITER_LEN  pContactMgrV20  242.05  gilda  0.000828
PCONTACTMGRV20_PID       pContactMgrV20  -0.12   gilda  42130
PCONTACTMGRV20_STATUS    pContactMgrV20  241.04  gilda  AppErrorFlag=false,Uptime=241.812,cpuload=0.2981,memory_kb=3224,memory_max_kb=3224,
```

Figure 2: **Example RealmCast Report:** A realmcast message is expanded into a multi-line report, consisting of a report on subscribed variables and published variables for a given application. In this case the report is for the `pContactMgrV20` application on vehicle `gilda`.

The content of the report can be adjusted by the client, e.g., `uMACViewer` user, to help visualize the important information. There are seven options:

- Source: The Source column of the report may be suppressed.
- Community: The Community column of the report may be suppressed.
- UTC Time: The time format may be shown in absolute UTC time, or relative to the start of the local MOOSDB.
- Subscriptions: The subscriptions portion of the report may be suppressed.

- Mask: In the subscriptions portion, virgin variables may be suppressed.
- Wrap: Long string content may be wrapped over several lines, e.g., as in Figure 2.
- Truncate: Long string content may be truncated.

Realmcast content may be adjusted through the seven buttons on the lower part of uMACViewer. These buttons are only present when realmcasting is enabled.

The start-up value of these settings may also be set to the user's liking in the uMACViewer configuration block:

```
realmcast_show_source        = true/false   // Default is true
realmcast_show_community     = true/false   // Default is true
realmcast_show_subscriptions = true/false   // Default is true
realmcast_show_masked        = true/false   // Default is true
realmcast_wrap_content       = true/false   // Default is false
realmcast_trunc_content      = true/false   // Default is false
realmcast_time_format_utc    = true/false   // Default is false
```

2.3 Publications and Subscriptions

2.3.1 Variables Published by uMACViewer

- APPCAST: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility.
- APPCAST_REQ_<COMMUNITY>: As an appcast viewer, pMarineViewer also generates outgoing appcast requests to MOOS communities it is aware of, including its own MOOS community. These postings are typically bridged to the other named MOOS community with the variable renamed simply to APPCAST_REQ when it arrives in the other community.

2.3.2 Variables Subscribed for by uMACViewer

- APPCAST: As an appcast enabled *viewer*, pMarineViewer also subscribes for appcasts from other other applications and communities to provide the content for its own viewing capability.
- REALMCAST: As an realmcast enabled *viewer*, pMarineViewer also subscribes for realmcasts from MOOS communities running pRealm, and renders them in the infocast panes described in Section **??**.
- WATCHCAST: As an realmcast enabled *viewer*, uMACViewer also subscribes for watchcasts from MOOS communities running pRealm, and renders them in the infocast panes.

2.4 Configuration File Parameters

A few configuration parameters are exposed to facilitate visual preference settings that would otherwise need to be done each time the application is launched via pull-down menus. These parameters are listed below, but may be recalled anytime by typing on the command line: `"uMACView -e"`.

Note: uMACView was released in 2012. Starting with the first release after 2019's Release 19.8, this app was augmented to include realmcasting in addition to appcasting. The term *infocasting* is the general term referring to either appcasting or realmcasting. As such, some of the earlier configuration parameters, e.g., appcast_font_size, have been replace with a term like infocast_font_size. The older configuration parameters are deprecated but still supported foreseeable releases.

Listing 2.1: Configuration Parameters for uMACView.

appcast_color_scheme:	Possible settings, `default`, `beige`, `indigo` or `white`. The default is `indigo`.
content_mode:	Sets the on startup content, which can be changed by the user at any time. Either `appcast` or `realmcast`. The default is `appcast`.
infocast_font_size:	Possible settings, `xsmall`, `small`, `medium`,`large`,`xlarge`. The default is `medium`.
infocast_height:	Possible settings, [`30,35,40,..., 85,90`]. The default is `70`.
nodes_font_size:	The font size used in the *nodes* pane of the set of infocasting panes. Possible settings, `xsmall`, `small`, `medium`, `large`, or `xlarge`. The default is `large`.
procs_font_size:	Possible settings, `xsmall`, `small`, `medium`, `large` or `xlarge`. The default is `large`.
realmcast_color_scheme:	Either `indigo`, `beige`, `hillside`, `white`, or `default`. The default is `hillside`.
realmcast_show_source:	If `true`, the Source column is shown on realmcast output. Setting to `false` conserves screen space, possibly enhancing readability. The default is `true`.
realmcast_show_community:	If `true`, the Community column is shown on realmcast output. Setting to `false` conserves screen space, possibly enhancing readability. The default is `true`.
realmcast_show_subscriptions:	If `true`, the Subscriptions block is shown on realmcast output. Setting to `false` conserves screen space, possibly enhancing readability. The default is `true`.
realmcast_show_masked:	If `true`, certain variables are excluded in realmcast output, e.g., virgin variables. Setting to `false` conserves screen space, possibly enhancing readability. The default is `true`.
realmcast_wrap_content:	If `true`, the variable Value column in realmcast output will wrap onto several lines. Setting to `true` possibly enhances readability for very long output. The default is `false`.
realmcast_trunc_content:	If `true`, the variable Value column in realmcast output will be truncated. Setting to `true` possibly enhances readability for very long output. The default is `false`.
realmcast_time_format_utc:	If `true` the Time column in realmcast output will be shown in UTC time instead of local time since app startup. The default is `false`.

refresh_mode: Possible settings, paused, events, streaming. The default is events.

The infocast_height refers to the relative height of the infocast pane to the window. The default of 70 means the pane will be 70% of the overall window height.

The refresh_mode refers to the policy of refreshing appcasts via appcast requests sent to the nodes and their applications. This is discussed below in Section 2.6

2.5 Command Line Arguments and Options

A few command line arguments are available. They are similar to most other MOOS-IvP applications. These arguments are listed below, but may be recalled anytime by typing on the command line:

```
$ MACView --help or -h
```

- --alias=<ProcessName>: Launch with the given process name rather than uMACView.
- --example, -e: Display example MOOS configuration block.
- --help, -h: Display command line usage.
- --interface, -i: Display MOOS publications and subscriptions.
- --version, -v: Display release version information.

2.6 Refresh Modes

The uMACView utility, operates in one of three *refresh modes*. This mode is always shown on the upper right in the title bar in reverse color. The three modes are the *streaming*, *events*, and *paused* modes.

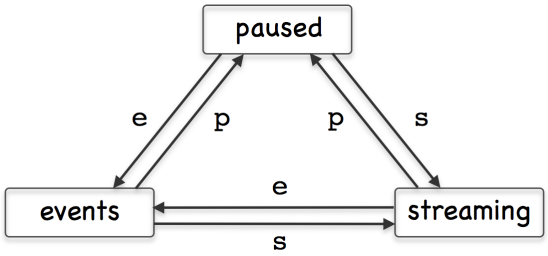

Figure 3: The uMAC utility is in one of three *refresh modes*, determining the manner in which appcast requests are conveyed to known applications and nodes. Reserved keyboard keys are used to transition between modes.

The Paused Refresh Mode In the *paused* refresh mode, no appcasts are solicited by the uMAC utility. The information being rendered should remain constant, virtually paused. Since appcast requests have a duration associated with them, prior appcast requests received by an application may need some time before expiring. Therefore the uMAC user may still see a trickle of updates after entering the paused mode. Furthermore, if there is another uMAC utility open, generating appcast requests, updates might still be seen after pausing.

The Events Refresh Mode In the *events* refresh mode, appcast requests of two types are sent to all known nodes and applications. The first type of request is sent only to the selected node and application. This type requests a continual update of appcasts, unconditionally. This application is, after all, the selected application for viewing. The second type of request is sent to all other nodes and applications requesting an appcast *only if a new run warning is generated*. The *events* refresh mode is the default mode upon launch.

The Streaming Refresh Mode In the *streaming* refresh mode, appcast requests are sent to all nodes and all applications, all the time. Furthermore, the request type is unconditional, meaning the application is requested to post a new appcast regardless of whether anything has changed in the application status. This mode is normally used for brief debugging, perhaps to check whether a node or application fails to respond. It creates a lot of appcast messaging traffic. It is not recommended to operate in this mode normally.

3 The uMAC Utility

The `uMAC` utility is an appcast viewer implemented in the terminal console. It acts the same as `uMACView` in terms of soliciting appcasts by publishing `APPCAST_REQ` messages and receiving `APPCAST` mail. The advantage over the GUI tool is the ability to remotely log into a vehicle and launch `uMAC` in a terminal window. This is espcially useful if the vehicle is otherwise not behaving in its communication with a shoreside MOOS community. Like `uMACView`, multiple versions of the utility may be running at the same time without interference with one another.

3.1 Content Modes

The viewable information in the `uMAC` tool is rendered in one of four *content modes*. Besides a *help* mode, the other three modes correlate to one of the three panes of the `uMACView` window in Figure 1. The primary content mode is the *appcast content mode*, where the output is an appcast of a particular application as shown in Figure 4.

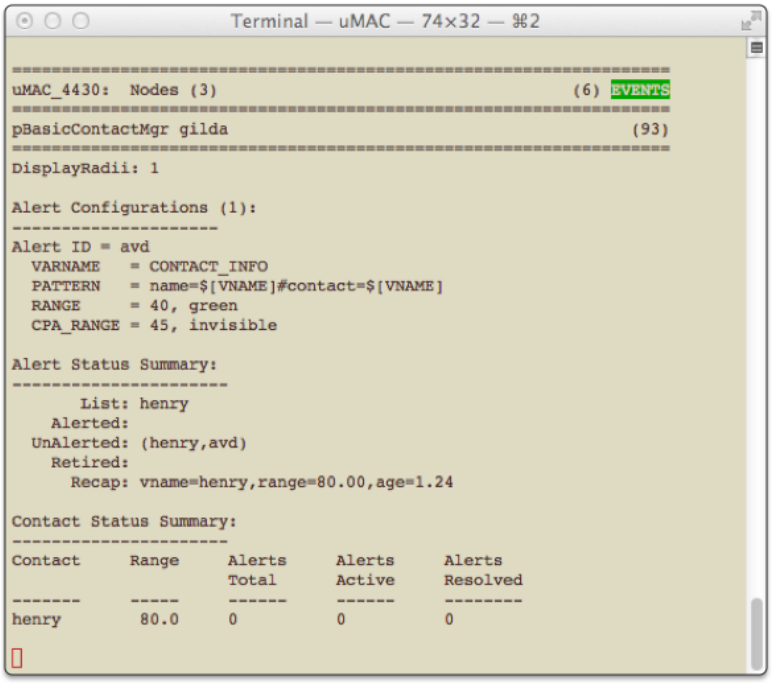

Figure 4: The uMAC utility monitors appcasts from a terminal window. The user may switch between appcasting sources by navigating with a keyboard menu. The primary advantage or uMAC is the ability to run it on a remotely deployed vehicle with a network connection.

The content of this window should look very similar to the bottom pane of the uMACView window in Figure 1. Two other content modes are supported, the *nodes content mode*, and the *procs content mode*. The former allows the user to select between different nodes, i.e., vehicles. The latter allows selection between different processes (MOOS applications) for the selected node. These modes correlate to the top two panes in the uMACView tool shown in Figure 1. A fourth, help, content mode is also supported. Transitioning between modes usually is done by selecting one of the choices presented, or popping back up a mode to allow a higher level choice. This done with three reserved keyboard keys, `p, n, h`, implementing the transitions shown in Figure 5.

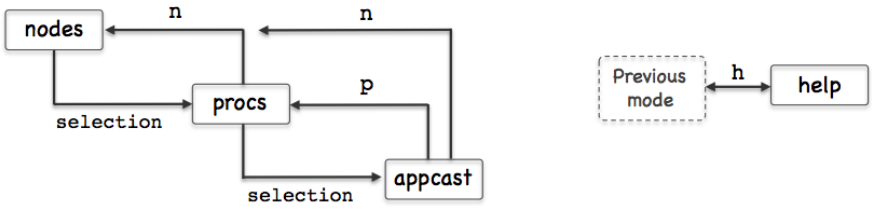

Figure 5: The uMAC utility monitors appcasts from a terminal window. The

An example rendering of uMAC in the *nodes content mode* is shown on the left in Figure 6. Each discovered node is assigned a character ID in the left-most column for selecting the node. The ID's are assigned as the nodes are discovered from incoming APPCAST messages.

The title line at the top of the report shows, on the left, the name of the uMAC application as it is known to the MOOSDB, and the number of nodes discovered. When uMAC is launched it gives itself a random suffix, such as uMAC_5991 in this example, to allow multiple uMAC sessions connect with the same MOOSDB. Recall the MOOSDB requires unique names across applications. The righthand side of the title line shows the number of iterations of the uMAC in parentheses, and the *refresh mode* shown in reverse color.

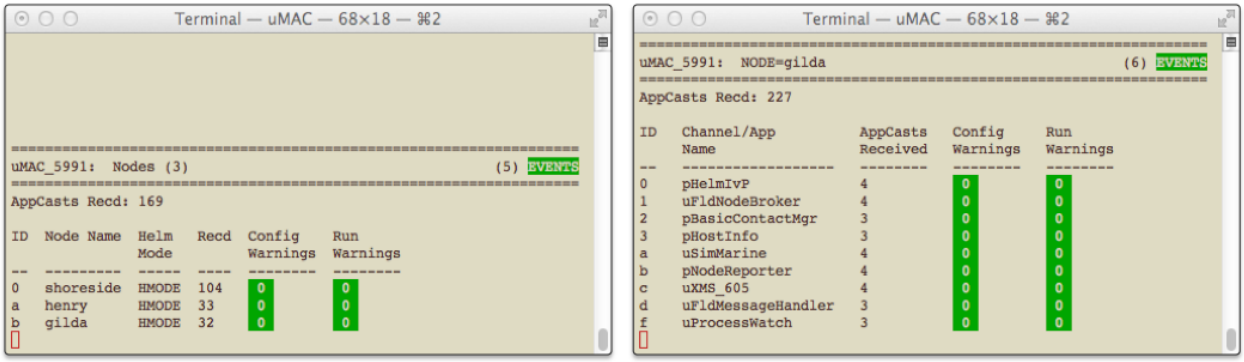

Figure 6: The **uMAC** utility monitors appcasts from a terminal window. The user may switch between appcasting sources by navigating with a keyboard menu. The primary advantage or **uMAC** is the ability to run it on a remotely deployed vehicle with a network connection.

An example from the *procs content mode* is shown on the right in Figure 6. Each discovered process (MOOS application) is assigned an ID in the left-most column for selecting the process. The ID's are assigned as the apps are discovered from incoming APPCAST messages. The title line at the top is nearly the same format as in the *nodes* content mode, except that the selected node is shown rather than the total nodes. The appcast counter just below the title line indicates the total number of appcasts received over all nodes, not just the selected node.

3.2 Refresh Modes

The uMAC utility, like the uMacView utility, operates in one of three *refresh modes*. This mode is always shown on the upper right in the title bar in reverse color. The three modes are the *streaming*, *events*, and *paused* modes. The desciption of these modes, given in Section 2.6, is also applicable to the operation for the uMAC utility.

3.3 A Tip Regarding Process Monitoring and uMAC Sessions

The uProcessWatch application is a utility for monitoring the presence of applications connected to the MOOSDB. It is appcast enabled, and will post a run warning when it detects the disappearance of a prior noted process. If a uMAC is launched and then exited, the uProcessWatch utility may interpret this as a problem and post a run warning. The effect may be that real run warnings are then later ignored. Tip: Add the following configuration line to uProcessWatch to ignore the exit of a uMAC session: `nowatcth = uMAC*`.

3.4 Publications and Subscriptions

The sole subscription for uMACView is the appcast message in the MOOS variable APPCAST. The sole publication is the appcast request, in the variable APPCAST_REQ.

3.5 Configuration File Parameters

There are no configuration file parameters specific to uMAC.

3.5.1 Command Line Arguments and Options

A few command line arguments are available. They are similar to most other MOOS-IvP applications. These arguments are listed below, but may be recalled anytime by typing on the command line:

```
$ uMAC --help or -h
```

- `--alias=<ProcessName>`: Launch with the given process name rather than uMACView.
- `--example, -e`: Display example MOOS configuration block.
- `--help, -h`: Display command line usage.
- `--interface, -i`: Display MOOS publications and subscriptions.
- `--version, -v`: Display release version information.

4 The uMACView Utility Integrated with pMarineViewer

The final uMAC tool is essentially uMACView embedded in the pMarineViewer application as shown in Figure 7. This is mostly just a convenience for users already using pMarineViewer. The appcasting mode may be toggled with the 'a' key to return to the traditional viewing layout. The AppCasting pull-down menu offers the same set of selections as uMACView with the exception of the hot keys used to switch between appcasting refresh modes since those keys were already used for other things in pMarineViewer.

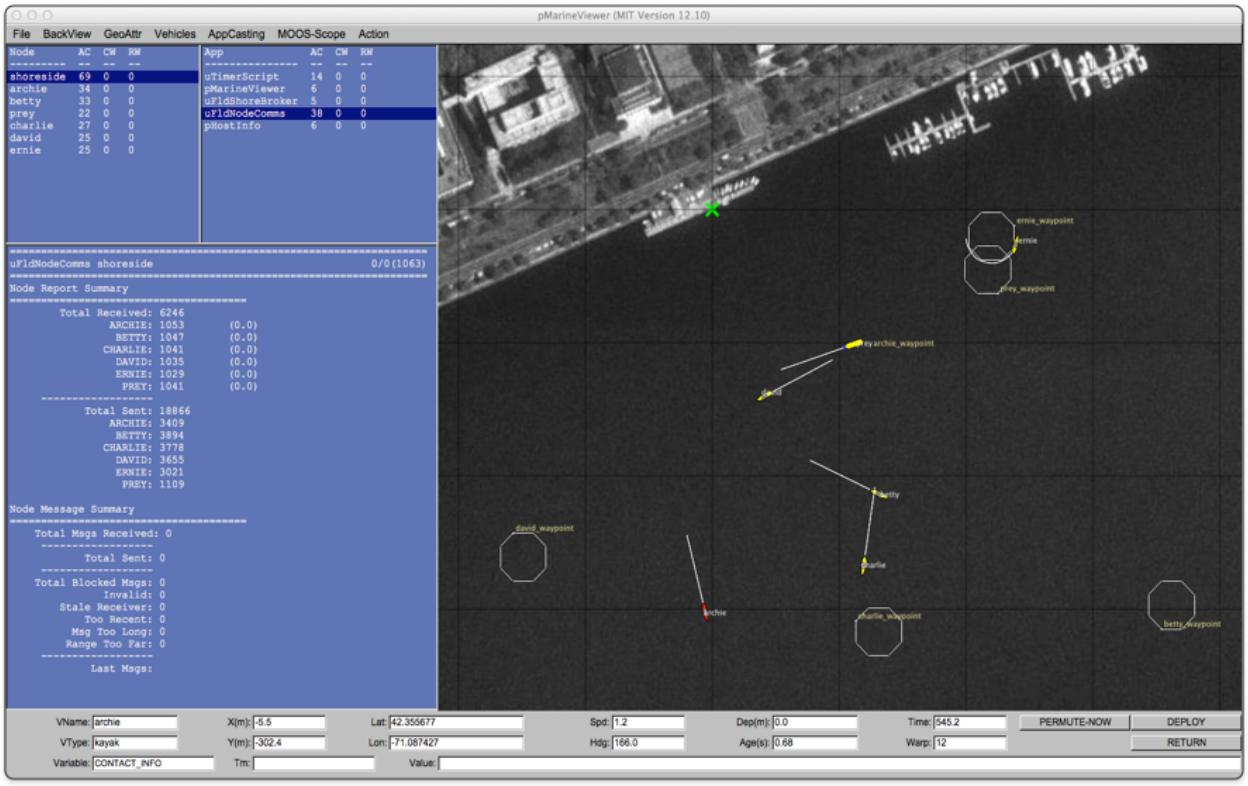

Figure 7: The pMarineViewer utility has an appcasting viewing capability very similar to uMACView embedded in the viewer. The rendering of the appcasting panes may be toggled on/off with the 'a' key.

In addition to toggling on/off the appcasting portion of the window, the width of the set of appcasting panes may be made wider or thinner using the CTRL-ALT-ARROW keys. By default the appcasting panes consume 30% of the width. The default height of the appcasting (bottom) portion of the appcasting panes consume 75% of the height of the set of appcasting panes. These startup extents may be changed with configuration the parameters:

```
appcasting_width  = 25    // legal values [20, 25,... ,65, 70]
appcasting_height = 80    // legal values [30, 35,..., 85, 90]
```

The prevailing value of these parameters can always be discovered by checking which radio button in the pull-down menu is presently selected.

References

[1] Michael R. Benjamin. pRealm: Integrated Scoping of the MOOSDB. http://oceanai.mit.edu/ivpman/apps/pRealm.

Enabling a MOOS Application for AppCasting

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1 Overview

In this section we discuss the steps for enabling a new or existing MOOS application to support appcasting. Much of the requisite appcasting source code is the same for any application. This is captured in a new `AppCastingMOOSApp` class to minimize appcasting boilerplate code. This class, as indicated in Figure 1, is a subclass of the common `CMOOSApp` class distributed with core MOOS.

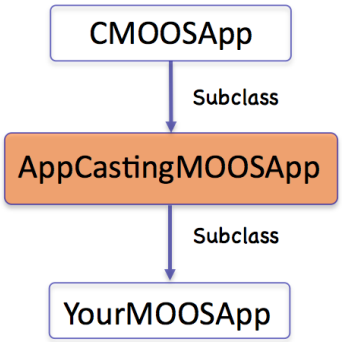

Figure 1: On-demand appcasting is implemented in a new CMOOSApp subclass called AppCastingMOOSApp.

The following is the complete list of required steps:

- Subclass the `AppCastingMOOSApp` superclass,
- Invoke a pair of superclass methods in the `Iterate()` function,
- Invoke a superclass method in the `OnNewMail()` function,
- Invoke a superclass method in the `OnStartUp()` function,
- Invoke a superclass method when registering for variables,
- Implement a `buildReport()` function where appcasts are formed.

In our discussions to follow, a hypothetical `YourMOOSApp` application and class definition is described. The above steps are the minimum requirements for appcasting, and they are fairly boilerplate, in most cases a single line of code. To make good use of the appcasting features, configuration warnings, run warnings and events may be placed in your application code. This is neither mandatory nor boilerplate, but really dependent on the application. Nevertheless, a few rules of thumb discussed:

- Posting events,
- Posting run warnings,
- Posting configuration warnings.

[2]

2 Sub-classing the AppCastingMOOSApp Superclass

The first step is to make `YourMOOSApp` a subclass of the `AppCastingMOOSApp`. This brings your application class everything from the traditional `CMOOSApp` class as well as the appcasting features of the `AppCastingMOOSApp` class. The only additional thing besides declaring the superclass is to declare the `buildReport()` function. This virtual function is invoked when an appcast has been deemed warranted. Appcasts are not typically generated on each iteration. See the later discussion about *on-demand appcasting*. The contents of the `buildReport()` function are discussed in greater detail in Section 7.

Listing 2.1: Pseudocode for sub-classing the AppCastingMOOSApp superclass.

```
1  #include "MOOS/libMOOS/Thirdparty/AppCasting/AppCastingMOOSApp.h"
2
3  class YourMOOSApp : public AppCastingMOOSApp        // Instead of CMOOSApp
4  {
5    // All your normal class declaration stuff
6
7    bool  buildReport();                              // Add this line
8  };
```

3 Invoking Superclass Methods in the Iterate() Method

The next step is to implement `YourMOOSApp::Iterate()` to invoke two superclass functions; one at the very beginning and one at the very end. The first superclass function, on line 2 below, does certain

common bookkeeping such as incrementing the counter representing the number of application iterations, and updating a variable holding the present MOOS time. The second superclass function, on line 6, invokes the on-demand appcasting logic discussed previously. If an appcast is deemed warranted, it will invoke the `buildReport()` function.

Listing 3.2: Pseudocode for invoking subclass methods in the `Iterate()` *method.*

```
1  bool YourMOOSApp::Iterate()
2  {
3    AppCastingMOOSApp::Iterate();                  // Add this line
4
5    // Do all your normal Iterate stuff
6
7    AppCastingMOOSApp::PostReport();               // Add this line
8    return(true);
9  }
```

4 Invoking a Superclass Method in the OnNewMail() Method

The next step is to implement `YourMOOSApp::OnNewMail()` to invoke a superclass function to have the first opportunity to handle incoming mail. For example, `APPCAST_REQ` mail is handled in the superclass. The list of mail messages is passed by reference to the superclass handler, allowing the `AppCastingMOOSApp::OnNewMail()` function to remove handled messages before returning to the mail handling implemented in `YourMOOSApp::OnNewMail()`.

Listing 4.3: Pseudocode for invoking a superclass method in the `OnNewMail()` *method.*

```
1  bool YourMOOSApp::OnNewMail(MOOSMSG_LIST &NewMail)
2  {
3    AppCastingMOOSApp::OnNewMail(NewMail);        // Add this line
4
5    // Do all your other normal mail handling.
6  }
```

5 Invoking a Superclass Method in the OnStartUp() Method

The next step is to implement `YourMOOSApp::OnStartUp()` to invoke a superclass function to perform startup steps needed by the `AppCastingMOOSApp` superclass.

Listing 5.4: Pseudocode for invoking a superclass method in the `OnStartUp()` *method.*

```
1  void YourMOOSApp::OnStartUp()
2  {
3    AppCastingMOOSApp::OnStartUp();               // Add this line
4
5    // Do all your other startup stuff
6  }
```

6 Invoking a Superclass Method When Registering for Variables

The next step is to invoke the `AppCastingMOOSApp::RegisterVariables()` wherever variables are registered in `YourMOOSApp` implementation. Many application developers have, in practice, created a

dedicated `registerVariables()` function, typically invoked at the conclusion of both `OnStartUp()` and `OnConnectToServer()`. The following example is one way to handle this.

Listing 6.5: Pseudocode for invoking a superclass method when registering for variables.

```
1  void YourMOOSApp::registerVariables()
2  {
3    AppCastingMOOSApp::RegisterVariables();      // Add this line
4
5    // Do all your other registrations
6  }
```

7 Implementing a buildReport Method for Generating AppCasts

The `buildReport()` function is where appcasts are made! The action that happens here is unique to the application. The form is designed by the application developer to reflect the most meaningful, concise snapshot of the application's present status. Recall that it is invoked automatically when or if the application deems an appcast is to be generated. For the purposes here though, a decision has indeed been made to generate an appcast, and `buildReport()` has been invoked to see that it happens. A simple example of `buildReport()` is shown below in Listing 6

Listing 7.6: Pseudocode for a very simple `buildReport()` example.

```
1  bool YourMOOSApp::buildReport()
2  {
3     m_msgs << "Number of good messages: " << m_good_message_count << endl;
4     m_msgs << "Number of bad  messages: " << m_bad_message_count  << endl;
5
6     return(true);
7  }
```

This simple example would generate something similar to the appcast rendered in Figure 2.

```
================================================
pYourMOOSApp alpha                        0/0  (97)
================================================
 # of good messages: 22
 # of bad  messages: 11
```

Figure 2: The rendering of a very simple appcast with just two message lines, no warnings, and no events. The header bar shows the name of the application, the originating MOOS community, the number of configuration and run warnings, and the application's current iteration counter.

Assume for the sake of the example that the two counter variables at the end of lines 2 and 3 are member variables for the fictitious `YourMOOSApp` class. The member variable `m_msgs` however is an STL stringstream also declared in the `AppCastingMOOSApp` class, for holding message output. The primary means of building an appcast is to add successive lines to the appcast via:

```
m_msgs <<   <element>   << endl;
```

where `<element>` may be a `string`, `double`, `int`, `unsigned int`, or any combination joined by the `"<<"` operator. A new line is indicated by tacking on the newline, `"\n"`, at the end. That's pretty much it, but there are a few other noteworthy points:

- Run warnings, configuration warnings, and events are not added during `buildReport()`, though not strictly prevented. They are more typically added as warnings are discovered, or events occur during the normal mail handling or iterate cycle.
- The header lines shown in Figure 2 is made automatically by the uMAC tool. They grab information in the appcast such as the application name and iteration number. These are filled by the boilerplate function calls such as `AppCastingMOOSApp::Iterate()` described earlier.
- The appcast is automatically cleared prior to each invocation of `buildReport()`. Warnings and events however are not cleared as discussed earlier.
- Returning true indicates that the appcast was indeed populated. Returning false would result in the appcast not being sent to the terminal or published to the MOOSDB. This is useful for some applications that may want to apply additional criteria before deciding to appcast.
- Although report formatting, e.g., columns or tables, is not natively supported somehow in the `buildReport()` routine, there are tools available that facilitate formatting that work easily with the `buildReport()` interface. They are discussed later in Section 99.

8 Posting Events

Events are messages (strings) that the application developer deems to be noteworthy enough to want to include in appcast output. An event may be posted anywhere in the application code with the `reportEvent()` function defined in the `AppCastingMOOSApp` class. A simple example:

```
reportEvent("Good msg received: " + message);
```

When the appcast is rendered in either a uMAC tool or in the terminal, the result would look similar to that in Figure 3.

```
================================================
pYourMOOSApp alpha                        0/1  (97)
================================================
RunTime Warnings: 14
 [11] Bad msg received: bricks

 # of good messages: 1
 # of bad  messages: 14

================================================
Most Recent Events (22):
================================================
[23.09] Good msg received: happyjoy
```

Figure 3: The rendering of a simple appcast with two message lines, a single run warning, and single event. The header bar shows the name of the application, the originating MOOS community, the number of configuration and run warnings, and the application's current iteration counter.

Recall that only a limited number of events are retained. Older events are dropped once a maximum amount is exceeded. The default event list size is eight, but this may be overridden for a particular application with the following parameter setting in the applications MOOS configuration block:

```
max_appcast_events = 25
```

The event list size may be set to at most 100.

9 Posting Run Warnings

Run warnings are similar to events, but they convey that something may have gone wrong. A run warning may be posted anywhere in the application code with the `reportRunWarning()` function defined in the `AppCastingMOOSApp` class. A simple example:

```
reportRunWarning("Bad msg received: " + message);
```

The appcast structure and uMAC tools are implemented such that run warnings are more easily brought to the attention of the operator. This is due to the following reasons:

- When an appcast has a run warning, the uMAC utilities will indicate so by turning the text red, as in Figure 4. Even when the focus of the uMAC utility is not on the particular appcast containing the run warning, the menu items for the appcast and vehicle will be rendered red. In Figure 4 for example, the menu browser focus is on vehicle `archie` and the `uFldMessageHandler` application. The red highlights also indicate there is a run warning on another application on `archie`, and there is also a run warning on vehicle `charlie`.
- An appcast request to an application may specify the reporting threshold to be `"run_warning"`. In this case an application will repeatedly choose not to publish an appcast unless a new run warning has been generated.
- The uMAC tools also keep a running tally of run warnings for each vehicle and each application under the column labeled `"RW"` as shown in Figure 4.

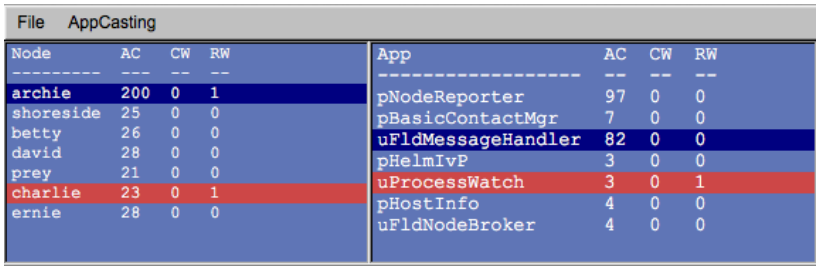

Figure 4: The uMAC tools will highlight an application that has produced an appcast with a run warning. If the run warning occurred on a node not currently in focus, e.g., charlie in the figure, the node itself is highlighted. The user can then select the other node to view the application generating the run warning.

Recall that only a limited number of run warnings are retained. Unlike events where the older ones are dropped once a maximum has been exceeded, old run warnings are never dropped. After the maximum has been reached, the generic warning `"Other Run Warnings"` is simply incremented. The default list size is ten, but this may be overridden for a particular application with the following parameter setting in the applications MOOS configuration block:

```
max_appcast_run_warnings = 50
```

The run warning list size may be set to at most 100.

10 Posting Configuration Warnings

Configuration warnings are similar to run warnings but they are typically only posted during the application startup, when the mission configuration file is read. A configuration warning may be posted with the `reportConfigWarning()` function defined in the `AppCastingMOOSApp` class. A simple example:

```
reportConfigWarning("Problem configuring FOOBAR. Expected a number but got: " + str);
```

There is a second way to post a configuration warning. This second method takes as an argument the original full configuration parameter line found in the mission file. Before posting the configuration warning it checks to see if the parameter was something that likely was handled by the superclass. This prevents the application from reporting that `AppTick=10"` is an unknown parameter for example.

```
reportUnhandledConfigWarning(original_full_config_line);
```

The appcast structure and uMAC tools are implemented such that configuration warnings are more easily brought to the attention of the operator. This is due to the following reasons:

- When an appcast has a configuration warning, the uMAC utilities will indicate so by turning the text green, as in Figure 5. Even when the focus of the uMAC utility is not on the particular appcast containing the config warning, the menu items for the appcast and vehicle will be rendered green. In Figure 5 for example, the menu browser focus is on the `shoreside` and the `uTimerScript` application application. The green highlights indicate there is a configuration warning in the `uFldShoreBroker` application and on other applications on `archie`, `charlie`, and `ernie`.
- The uMAC tools also keep a running tally of configuration warnings for each vehicle and each application under the column labeled `"CW"` as shown in Figure 5.

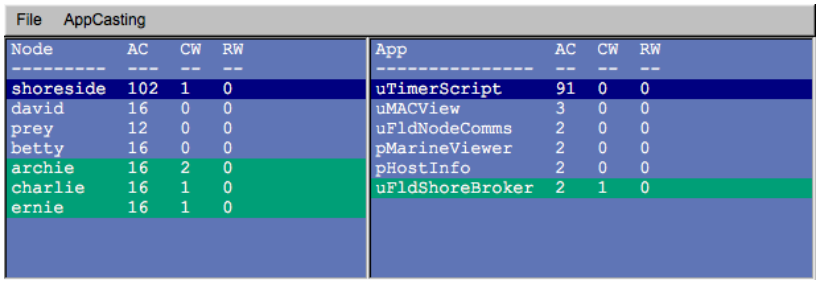

Figure 5: The uMAC tools include will highlight an application that has produced an appcast with a configuration warning. If the configuration warning occurred on a node not currently in focus, e.g., archie, charlie, or ernie in the figure, the node itself is highlighted. The user can then select the other node to view the application generating the warning.

Like run warnings and events, configuration warnings are limited in number to prevent runaway growth in the size of an appcast over time. The limit however is large, 100, and fixed. Presumably the number of configuration warnings is limited by the number of possible configuration parameters for an application, and large number of configuration warnings usually indicates that a mission should be halted and fixed before moving on.

The example code in Listing 7 below is an example `OnStartUp()` method showing the intended scenarios of reporting configuration warnings.

Listing 10.7: Pseudocode example for `OnStartUp()` *configuration warning handling.*

```
1  bool YourMOOSApp::OnStartUp()
2  {
3    AppCastingMOOSApp::OnStartUp();
4
5    STRING_LIST sParams;
6    if(!m_MissionReader.GetConfiguration(GetAppName(), sParams))
7      reportConfigWarning("No config block found for " + GetAppName());
8
9    STRING_LIST::iterator p;
10   for(p=sParams.begin(); p!=sParams.end(); p++) {
11     string orig  = *p;
12     string line  = *p;
13     string param = toupper(MOOSChomp(line, "="));
14     string value = line;
15
16     if(param == "FOO") {
17       bool handled = handleConfigFOO(value);
18       if(!handled)
19         reportConfigWarning("Problem with configuring FOO: " + value);
20     }
21     else if(param == "BAR")
22       bool handled = handleConfigBAR(value);
23       if(!handled)
24         reportConfigWarning("Problem with configuring BAR: " + value);
25     }
26
27     else
28       reportUnhandledConfigWarning(orig);
```

```
29    }
30    return(true);
31  }
```

There are a few issues worth noting in this example:

- A check is made that the application actually has a configuration block. This is done on lines 5-6, and catches a common bug with newly minted mission files.
- Checks are made that known parameters have legal values. This is done for the parameters `FOO` in lines 15-19 and `BAR` in lines 20-24 in Listing 6. In each case an external handler is invoked, e.g., `handleConfigFOO(value)` on line 16, which returns a Boolean indicating whether the parameter value was proper or not. If not, a configuration warning is reported as on lines 18 and 23.
- A final case is handled (lines 26-27) if the present parameter is not matched by any of the previous cases. This catches the common mistake of mis-spelling the parameter name. The `reportUnhandledConfigWarning()` function is used rather than the `reportConfigWarning()` function. The former function takes the whole original configuration line as input and checks to see if the parameter was a parameter handled at the superclass level. This prevents the generation of a warning for a line like `AppTick=5`.

Under the Hood of On-Demand AppCasting

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1 Overview

On-demand appcasting refers to the goal of minimizing generated appcasts, ideally only when there is a reasonable chance that an appcast will be tended to (looked at) by a user. Users may be reading an appcast either by looking at terminal output or through a uMAC utility.

2 Motivation

Consider the following scenario:

- 20 simulated vehicles,
- 10 MOOS applications on each vehicle,
- Each application running with an apptick of 4Hz,
- MOOS time warp running at 25x real time.

In the above scenario (a conceivable scenario in our lab), $20,000$ appcasts would be generated *per second*. Consider a second scenario:

- One fielded underwater vehicle,
- 10 MOOS applications,
- Each application running with an apptick of 4Hz,
- Mission duration 6 months.

Although only 40 appcasts per second are generated, the vehicle is underwater and, limited to acoustic messages, likely no appcast will ever be viewed. With a six month mission, the CPU time and power budget needed to generate those 40 reports per second may come under scrutiny.

3 AppCast Generation Criteria

An appcast will be generated on any given iteration only if the contingencies and criteria depicted in Figure 1 are met. In short, an appcast is generated if either a terminal is open or a uMAC tool is requesting the appcast. Even when appcasts are being generated, they may be generated less frequently than each iteration, to be more in line with the frequency a user is able to process refreshed report information. These issues are discussed next.

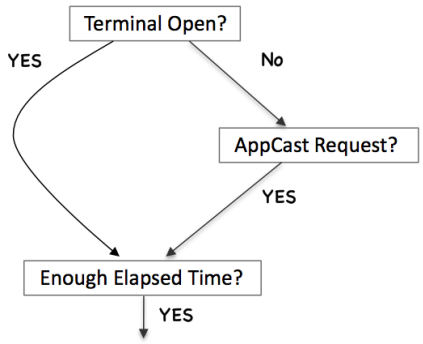

Figure 1: The appcast-generation decision is based on a few checks and contingencies. An appcast is generated only when tended to by a user, and only as often as a user may reasonably expect to process updated reports.

4 Terminal Switching

In the flow diagram of Figure 1, the first step in determining whether an appcast is to be generated involves the presence of a terminal. If an app is launched with a terminal window open, or launched within a terminal window, appcasts should be generated. In this regard, appcasting apps should behave like non-appcasting apps, although the former produce its output by different means. From the user's perspective, launching an application with or within a terminal window should result in the same familiar behavior regardless of the application.

Upon startup an AppCastingMOOSApp application will try to determine if information directed to `stdout` will make its way to an open terminal window. The following reasoning is applied:

- By default the application assumes information directed to `stdout` is indeed being rendered in a terminal window.
- During startup, when mission file parameters are examined, if the application detects the presence of a global parameter, `TERM_REPORTING`, and it is set to `"false"`, then the application assumes that a terminal window is not open to receive output written to `stdout`.
- During application startup, the following library utility function is invoked:
  `int isatty(int);`
  This function is defined in `"unistd.h"` and is able to detect whether `stdout` is receiving output.

The above ensures that the application will err on the side of always producing appcasts/output to the terminal. To ensure otherwise, make sure to include `TERM_REPORTING="false"` in the MOOS configuration file. The last check is just a convenience or extra fail-safe; if an application is launched with `pAntler` using `NewConsole=false` and `pAntler` itself is also launched with `stdout` redirected to `/dev/null/`, then the application will automatically detect this. This style of launching is actually a fairly common scenario in launching MOOS communities on vehicles in our lab. In detecting this situation, the application will not generate an appcast unless a uMAC tool is explicitly requesting it. This is the next step in the flow diagram of Figure 1, and discussed next.

5 AppCast Requests

An *appcast request* is message sent to an application to begin or continue generating appcasts for some specified period of time. The request may originate in the local MOOSDB community, or in a remote MOOSDB community as the two cases suggest in Figure 2.

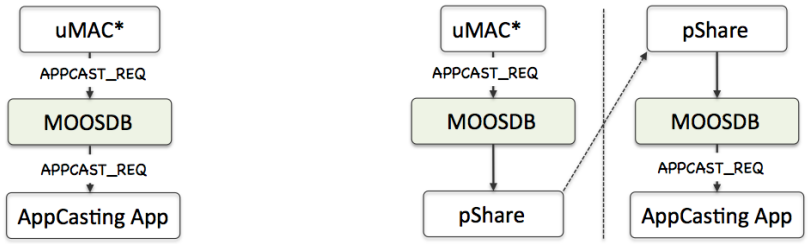

Figure 2: An appcast request requests that the receiving MOOS application begin or continue to generate appcasts for some specified period of time. The request may come from an app within the same MOOS community, or from an external community.

The content of an appcast request has the following fields:

- **node**: This must match the name of the MOOS community within which the application is running, otherwise the appcast request is ignored. If the `node` is `"any"`, the match is always granted.
- **app**: This must match the name of the MOOS application receiving the appcast request, otherwise the request is ignored. If the `app` is `"any"`, the match is always granted.
- **duration**: This number is given in seconds and represents the duration the appcast request would honored after time of receipt. The maximum value is 30.
- **threshold**: The threshold name indicates under what conditions the receiving application should generate an appcast. The two possible values are `"any"` and `"run_warning"`. Their meaning is discussed below.
- **key**: The key helps the receiving application discern appcast requests from different applications.

An example `APPCAST_REQ` message:

```
APPCAST_REQ = "node=henry,app=uProcessWatch,duration=3.0,key=pMarineViewer:shore,thresh=any"
```

Node and Application Name Matching

An appcast request will be ignored by a receiving application if the request does not *match*. The match must be made between both (a) the requested and actual node name, and (b) the requested and actual application name. A requested value `"any"` will match to anything. In most circumstances the requesting application, e.g., `uMAC`, `uMACView` or `pMarineViewer`, will publish requests naming nodes and applications explicitly. Upon startup however, requests may be sent broadly to all nodes and all applications just to learn about existing appcasting sources before beginning to make requests explicitly.

Duration Time

An appcast request comes with a duration. If the requesting application disconnects, the duration caveat ensures the original request will timeout before too long, avoiding the perpetual generation of now unwanted appcasts. Typically if a uMAC tool is monitoring the appcasts of a particular application, it repeatedly sends an appcast request to that application, each time refreshing the timeout criteria.

Request Threshold

The appcast request will specify one of two criteria to the application. First, if the criteria is `"any"`, the application is to publish an appcast on each possible occasion. This may not be the same as each iteration, since a further minimum reporting interval may be applied, as described next in Section 6. The second threshold type is `"run_warning"`. This indicates to the receiving application that it should only generate an appcast when a new run warning has been added since the last appcast. Typically a uMAC tool will run in a mode sending appcast requests with the latter threshold to virtually all nodes and apps, but appcast requests using the former threshold to a single node and app chosen by the user.

Request Key

AppCast requests specify a particular key, presumably a string that is unique to the originator. This allows the receiving application to do separate bookkeeping for each requesting application. As long as the appcast request threshold matches, hasn't timed out, and met the threshold criteria for *one* of its logged keys, then it indeed meets the criteria.

6 Limiting the AppCast Frequency

A third criteria is applied to the decision of generating an appcast on any given iteration. This criteria is depicted at the bottom of Figure 1. Even if a terminal window is open, or a valid appcast request has been received recently enough, the generation of an appcast may still be skipped if the previously generated appcast was generated too recently. AppCast content is meant to be human-readable, and humans can only read so fast. There is no sense in updating a terminal report say 50 times per second. Although most MOOS apps are typically not configured with an apptick of 50Hz, an apptick of 5 is fairly common, as well as a `MOOSTimeWarp` of say 10 or greater. By default, appcasting applications are limited in real-time frequency to once every 0.6 seconds.

This number just reflects a number that, from experience, feels right for an update frequency. This can be overridden for any application with the following parameter in its configuration block:

```
term_report_interval = N
```

where N ranges from zero (appcast generation only limited by the iteration frequency) to at most 30 seconds.

7 Generating and AppCast vs. Publishing and AppCast

The flow chart shown in Figure 1 addresses the issue of whether or not an appcast is *generated*. Whether or not the appcast is *published* is a separate issue. Simply put, if a terminal window is open for the application, and it has not received any appcast requests, the appcast is generated (and rendered to the terminal `stdout`) but not published.

Below is informal logic shorthand for policy and conditions described over the last few pages. First, on the policy of whether an appcast is generated:

```
generate_appcast = (terminal || appcast_requested) && !recent_appcast
```

Next, on the question of whether an appcast is published:

```
publish_appcast  = generate_appcast && appcast_requested
```

Both of the above depend on the term `appcast_requested` from the following expression:

```
appcast_requested = unexpired_request && ((request_threshold == "any") || new_run_warning)
```

The terms in this last expression will hopefully be apparent from the preceding pages.

8 Monitoring AppCast Traffic Volume

The uMAC tools provide a means for verifying that on-demand appcasting is behaving. These same tools are also useful for catching other anomalies. The information in Figure 3 below focuses on the information at the top of the `uMACView` tool depicted in Figure 3.

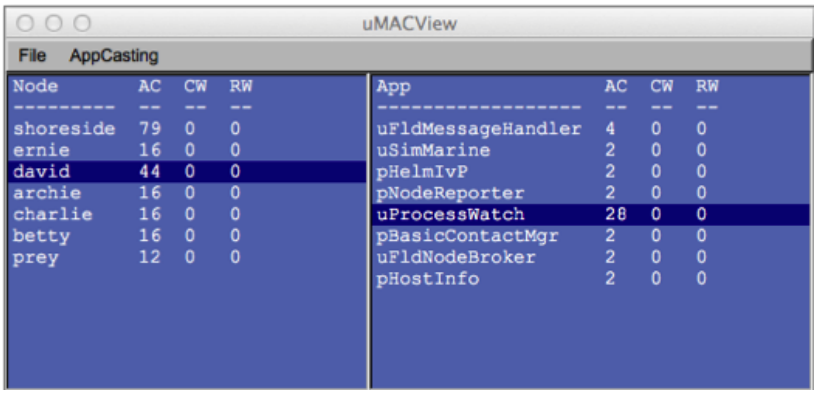

Figure 3: The uMAC tools include tallies of the number of appcasts received for each node (vehicle) and each application. The above is from the top of the `uMACView` and appcast tallies are shown under the `"AC"` column.

From the above figure, one might ask why so many appcasts have been received when the user is only focusing on one vehicle and one application. There are few answers to this question and each, listed below, are related to the need for a uMAC utility to *find* existing sources of appcasts. After all, how can it send an appcast request to a particular vehicle and particular application if it doesn't know that it exists?

- When an application starts up, it generates appcasts for the first few iterations. Even if no one is tending to this information, a few iterations worth of un-tended appcasts is not harmful. In practice this helps uMAC tools discover appcasting apps.
- When a uMAC tool starts up, it posts an appcast request to all known nodes and all known applications. The uMAC tool doesn't need to know or keep track of vehicles, but just simply posts `APPCAST_REQ="node=all,app=all, ..."` to its local MOOSDB. Other applications have the responsibility of bridging this variable to other vehicles as they are discovered.
- Each time a uMAC tool receives a new appcast from a vehicle/node it has never heard from before, it responds by posting an appcast request back to that vehicle for all applications, e.g.., `APPCAST_REQ="node=henry,app=all, ..."`. This request will time-out shortly, but is usually sufficient to learn about all applications on that node. Subsequent requests are then more selective.